



University of Asia Pacific

Department of Computer Science and Engineering

CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 5

Experiment Title:

Design, Implementation, Synthesis, and Simulation of an 8-Bit Arithmetic Logic Unit (ALU) Using Verilog HDL in Xilinx ISE

Submitted by:

Name: MD. Istiak Ahammed Nabil

Student ID: 22201133

Section: C-1

Submitted to:

Md. Rasheduzzaman

Lecturer

Department of Computer Science and Engineering

Date of Submission: 4/10/26

1 Title of the Experiment

Design, Implementation, Synthesis, and Functional Simulation of an 8-Bit Arithmetic Logic Unit (ALU) Using Verilog HDL in Xilinx ISE Design Suite.

2 Objectives

The objectives of this experiment are:

- To design an 8-bit Arithmetic Logic Unit (ALU) using Verilog Hardware Description Language (HDL).
- To implement arithmetic operations such as addition and subtraction.
- To implement logical operations including AND, OR, XOR, and NOR.
- To implement one-bit left and right shift operations.
- To generate and analyze the RTL schematic of the ALU using Xilinx ISE.
- To synthesize the Verilog design and observe its gate-level RTL representation.
- To verify the functional correctness of the ALU using the ISim simulator.
- To observe the behavior of the CarryOut and Zero status flags for different operations.

3 Theory

An Arithmetic Logic Unit (ALU) is a combinational digital circuit that performs arithmetic and logical operations on binary operands.

In this experiment, an 8-bit parameterized ALU is designed using Verilog HDL. The ALU accepts two 8-bit operands, A and B , and a 3-bit operation selector, ALU_Sel .

The ALU produces an 8-bit result, ALU_Out , together with two status signals:

- **CarryOut:** Indicates a carry generated during addition.
- **Zero:** Becomes high when the ALU output is zero.

The operation performed by the ALU depends on the value of ALU_Sel .

Table 1: ALU Operation Selection Table

<i>ALU_Sel</i>	Operation	Description
000	Addition	$A + B$
001	Subtraction	$A - B$
010	AND	$A \wedge B$
011	OR	$A \vee B$
100	XOR	$A \oplus B$
101	NOR	$\overline{A \vee B}$
110	Shift Left	$A \ll 1$
111	Shift Right	$A \gg 1$

The ALU is implemented as a combinational circuit using an `always @(*)` block. Therefore, the output responds immediately to changes in the input operands or operation selector.

4 Apparatus / Hardware & Software Requirements

- **Host PC:** Windows 10/64-bit environment
- **Virtualization Platform:** Oracle VM VirtualBox
- **Operating System Environment:** Oracle Linux 64-bit virtual machine
- **EDA Software:** Xilinx ISE Design Suite 14.7
- **Simulation Tool:** ISim
- **Hardware Description Language:** Verilog HDL

5 ALU Architecture

The designed ALU contains two 8-bit input operands, A and B , and a 3-bit operation selector, ALU_Sel .

The output consists of an 8-bit result ALU_Out , a carry flag $CarryOut$, and a zero flag $Zero$.

The general architecture can be represented as:

$$A[7 : 0], B[7 : 0], ALU_Sel[2 : 0] \longrightarrow \text{8-Bit ALU} \longrightarrow ALU_Out[7 : 0], CarryOut, Zero$$

The operation selector determines which arithmetic or logical operation is performed.

6 RTL-Schematic Diagrams

The RTL schematic was generated using Xilinx ISE after the Verilog ALU design was synthesized.

6.1 Top-Level RTL Schematic

The top-level RTL schematic represents the ALU as a single functional block with its input and output ports.

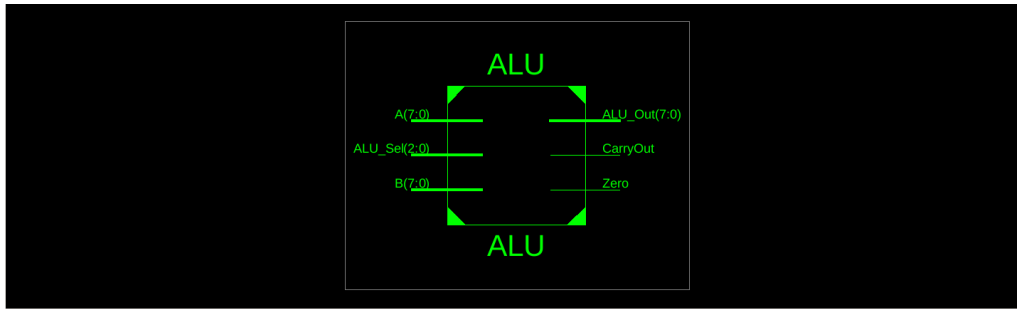


Figure 1: Top-level RTL schematic of the 8-bit ALU.

The schematic contains the following ports:

- $A[7 : 0]$ – First 8-bit operand.
- $B[7 : 0]$ – Second 8-bit operand.
- $ALU_Sel[2 : 0]$ – 3-bit operation selector.
- $ALU_Out[7 : 0]$ – 8-bit ALU result.
- *CarryOut* – Carry status output.
- *Zero* – Zero-result status output.

6.2 Internal RTL Schematic

The internal RTL representation shows the combinational logic responsible for selecting and generating the appropriate ALU operation.

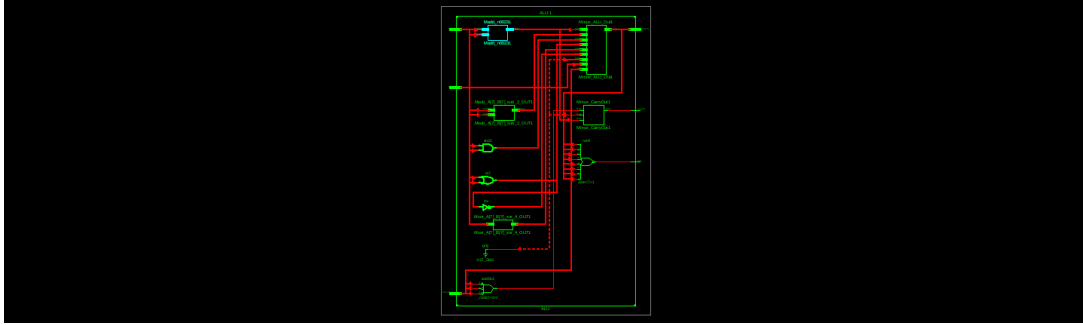


Figure 2: Internal RTL schematic of the 8-bit ALU.

6.3 Gate-Level RTL Schematic

After synthesis, Xilinx ISE expands the behavioral ALU description into the corresponding combinational hardware structure.

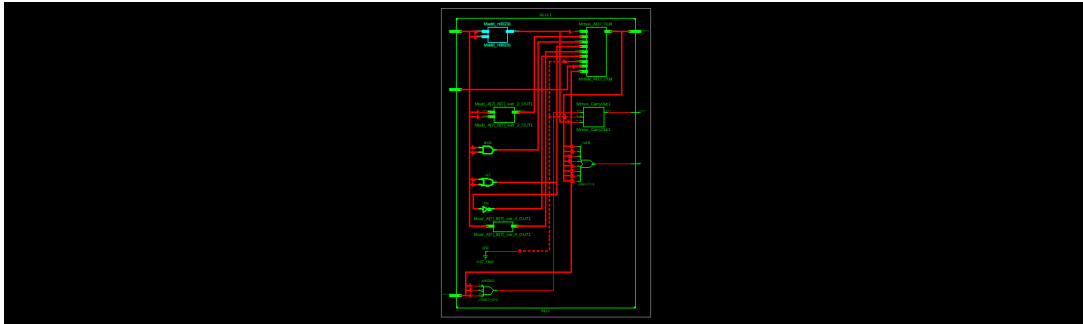


Figure 3: Synthesized gate-level RTL schematic of the ALU.

7 Verilog HDL Implementation

The ALU is implemented as a parameterized module with a default data width of 8 bits.

Listing 1: Verilog implementation of the 8-bit parameterized ALU.

```

1  `timescale 1ns / 1ps
2
3  module ALU #(parameter WIDTH = 8) (
4      input wire [WIDTH-1:0] A,
5      input wire [WIDTH-1:0] B,
6      input wire [2:0] ALU_Sel,
7      output reg [WIDTH-1:0] ALU_Out,
8      output reg CarryOut,
9      output wire Zero
10 );
11
12 always @(*) begin
13     CarryOut = 1'b0;

```

```

14
15     case(ALU_Sel)
16
17         3'b000:
18             {CarryOut, ALU_Out} = A + B;
19
20         3'b001:
21             ALU_Out = A - B;
22
23         3'b010:
24             ALU_Out = A & B;
25
26         3'b011:
27             ALU_Out = A | B;
28
29         3'b100:
30             ALU_Out = A ^ B;
31
32         3'b101:
33             ALU_Out = ~(A | B);
34
35         3'b110:
36             ALU_Out = A << 1;
37
38         3'b111:
39             ALU_Out = A >> 1;
40
41         default:
42             ALU_Out = {WIDTH{1'b0}};
43
44     endcase
45 end
46
47 assign Zero = (ALU_Out == {WIDTH{1'b0}});
48
49 endmodule

```

8 Testbench Implementation

A Verilog testbench was developed to verify all eight ALU operations.

The testbench applies different operands and selector values at different simulation times. The output, carry flag, and zero flag are monitored using the `$monitor` system task.

Listing 2: Verilog testbench used for ALU simulation.

```

1  `timescale 1ns / 1ps
2
3  module tb_ALU;
4
5      reg [7:0] A;
6      reg [7:0] B;
7      reg [2:0] ALU_Sel;
8
9      wire [7:0] ALU_Out;
10     wire CarryOut;
11     wire Zero;
12
13     ALU #(.WIDTH(8)) uut (
14         .A(A),
15         .B(B),
16         .ALU_Sel(ALU_Sel),
17         .ALU_Out(ALU_Out),
18         .CarryOut(CarryOut),
19         .Zero(Zero)
20     );
21
22     initial begin
23
24         $display("Time\t A \t B \t Sel\t Out\t C_out\t Zero");
25
26         $monitor("%0t\t %d\t %d\t %b\t %d\t %b\t %b",
27             $time, A, B, ALU_Sel,
28             ALU_Out, CarryOut, Zero);
29
30         A = 0;
31         B = 0;
32         ALU_Sel = 0;
33
34         #100;
35
36         // Test 1: Addition
37         A = 8'd15;
38         B = 8'd10;
39         ALU_Sel = 3'b000;
40         #20;
41
42         // Test 2: Addition with CarryOut
43         A = 8'd250;

```

```

44     B = 8'd10;
45     ALU_Sel = 3'b000;
46     #20;
47
48     // Test 3: Subtraction
49     A = 8'd45;
50     B = 8'd15;
51     ALU_Sel = 3'b001;
52     #20;
53
54     // Test 4: Subtraction producing Zero
55     A = 8'd50;
56     B = 8'd50;
57     ALU_Sel = 3'b001;
58     #20;
59
60     // Test 5: Bitwise AND
61     A = 8'b10101010;
62     B = 8'b11001100;
63     ALU_Sel = 3'b010;
64     #20;
65
66     // Test 6: Bitwise OR
67     A = 8'b10101010;
68     B = 8'b11001100;
69     ALU_Sel = 3'b011;
70     #20;
71
72     // Test 7: Bitwise XOR
73     A = 8'b11110000;
74     B = 8'b01010101;
75     ALU_Sel = 3'b100;
76     #20;
77
78     // Test 8: Shift Left
79     A = 8'b00001111;
80     B = 8'd0;
81     ALU_Sel = 3'b110;
82     #20;
83
84     #20;
85
86     $display("Simulation finished.");
87     $finish;
88

```

```

89     end
90
91 endmodule

```

9 Simulation

Functional simulation of the ALU was performed using the ISim simulator included with Xilinx ISE.

The testbench applies a sequence of arithmetic and logical operations to verify the behavior of the ALU.

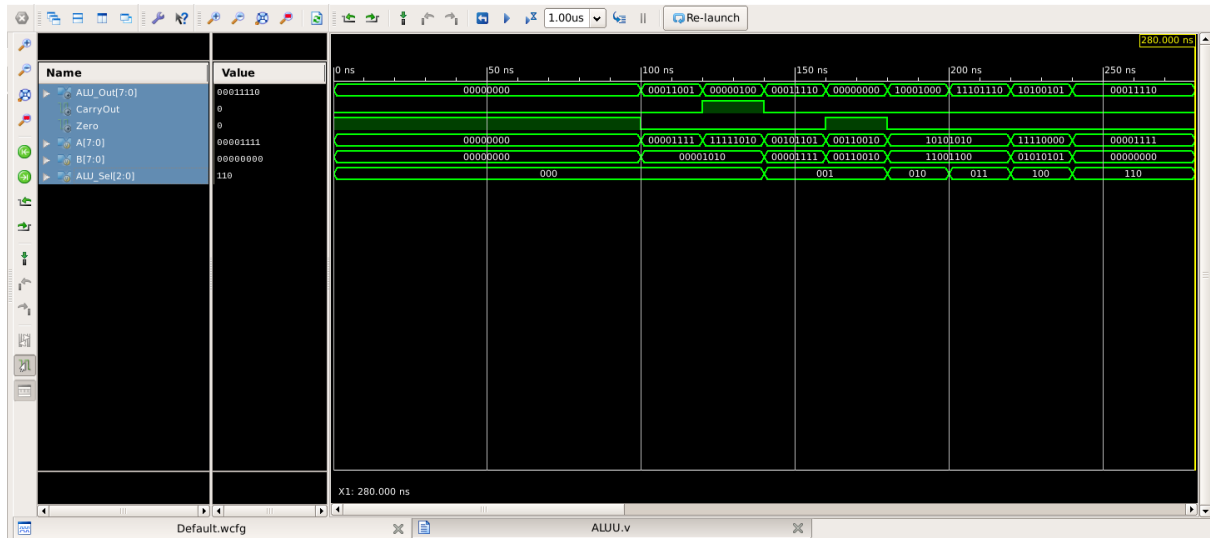


Figure 4: ISim simulation waveform of the 8-bit ALU.

9.1 Simulation Verification Table

The following table shows the expected outputs for the test cases implemented in the testbench.

Table 2: ALU Simulation Verification

No.	Sel	A	B	Operation	Expected Out	Carry	Zero
1	000	15	10	Addition	25	0	0
2	000	250	10	Addition	4	1	0
3	001	45	15	Subtraction	30	0	0
4	001	50	50	Subtraction	0	0	1
5	010	10101010	11001100	AND	10001000	0	0
6	011	10101010	11001100	OR	11101110	0	0
7	100	11110000	01010101	XOR	10100101	0	0
8	110	00001111	00000000	Shift Left	00011110	0	0

9.2 Output Analysis

The simulation verifies the major functions of the ALU.

- For the addition operation, $15 + 10 = 25$, producing an output of 00011001.
- For $250 + 10 = 260$, the 8-bit result wraps around to 00000100 and the CarryOut flag becomes high.
- For subtraction, $45 - 15 = 30$, producing 00011110.
- For $50 - 50 = 0$, the output becomes zero and the Zero flag becomes high.
- The AND operation produces $10101010 \text{ AND } 11001100 = 10001000$.
- The OR operation produces $10101010 \text{ OR } 11001100 = 11101110$.
- The XOR operation produces $11110000 \text{ XOR } 01010101 = 10100101$.
- The left-shift operation transforms 00001111 into 00011110.

Therefore, the simulation confirms that the ALU responds correctly to the selected operation codes.

10 Code in GitHub Repository

The Verilog HDL source code and testbench for this experiment will be maintained in a GitHub repository.

- **Repository Link:** <https://github.com/IstiakNabil/Vlsi-Lab-Report>

- **Design File:** `ALU.v`
- **Testbench File:** `tb_ALU.v`

11 Result

The 8-bit parameterized Arithmetic Logic Unit was successfully designed using Verilog HDL and synthesized using Xilinx ISE.

The RTL schematic confirmed the intended structure of the design. Functional simulation using ISim successfully verified arithmetic, logical, and shift operations.

The CarryOut flag was correctly generated during an addition that exceeded the 8-bit range, while the Zero flag correctly indicated a zero-valued ALU output.

12 Conclusion

In this experiment, an 8-bit Arithmetic Logic Unit was designed, implemented, synthesized, and functionally simulated using Verilog HDL and Xilinx ISE Design Suite.

The ALU was designed to perform eight different operations: addition, subtraction, AND, OR, XOR, NOR, left shift, and right shift.

The generated RTL schematic demonstrated the structural representation of the ALU, while the ISim simulation verified the functional behavior of the implemented operations.

The simulation also confirmed the correct operation of the CarryOut and Zero status flags.

Therefore, the objectives of designing, synthesizing, and simulating an 8-bit ALU using Xilinx ISE were successfully achieved.